

撿球機

2026/01/20

目錄

- 撿球方式選擇
- 影像辨識系統

馬達

空載轉速 (No-load Speed): 200 rpm

負載轉速 (Rated Speed): 175 rpm

單顆馬達額定扭力 (Rated Torque): 2.29 kgf. cm

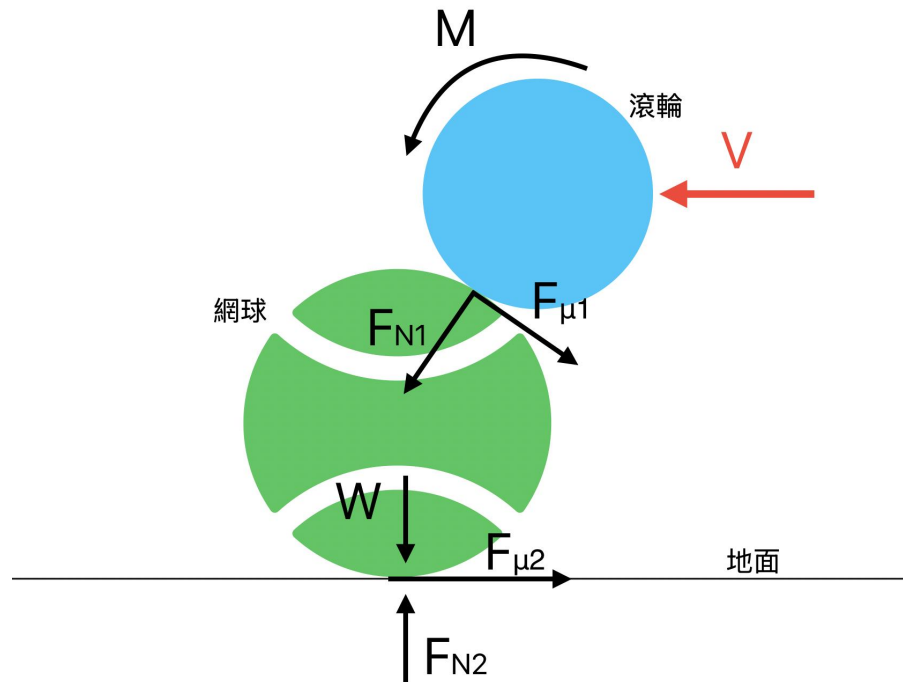
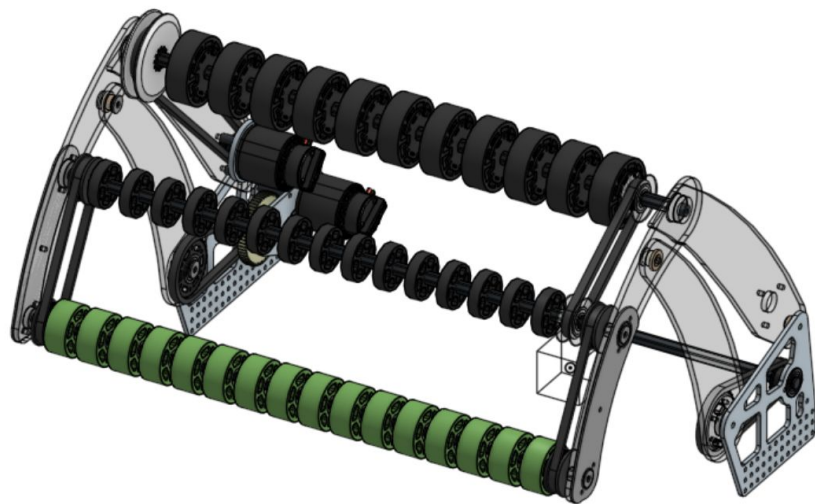
兩顆馬達總扭力: $2.29 \times 2 = 4.58$ kgf. cm

假設輪子直徑15 cm, 輪子周長: $15 \text{ cm} \times 3.14 = 47.1 \text{ cm}$, 以175 rpm做計算秒速約1.35 m/s

總推力= 扭力/半徑= $4.58 \text{ kgf. cm} / 7.5 \text{ cm} = 0.611 \text{ kgf}$

假設平地摩擦係數=0.046, 重量=推力/摩擦係數= $0.611 / 0.046 = 13.275 \text{ kg}$

上滾輪式撿球機構



上滾輪撿球機構計算

標準網球直徑: 67mm

網球剛性: 12550N/m

標準網球重量: 57.7g

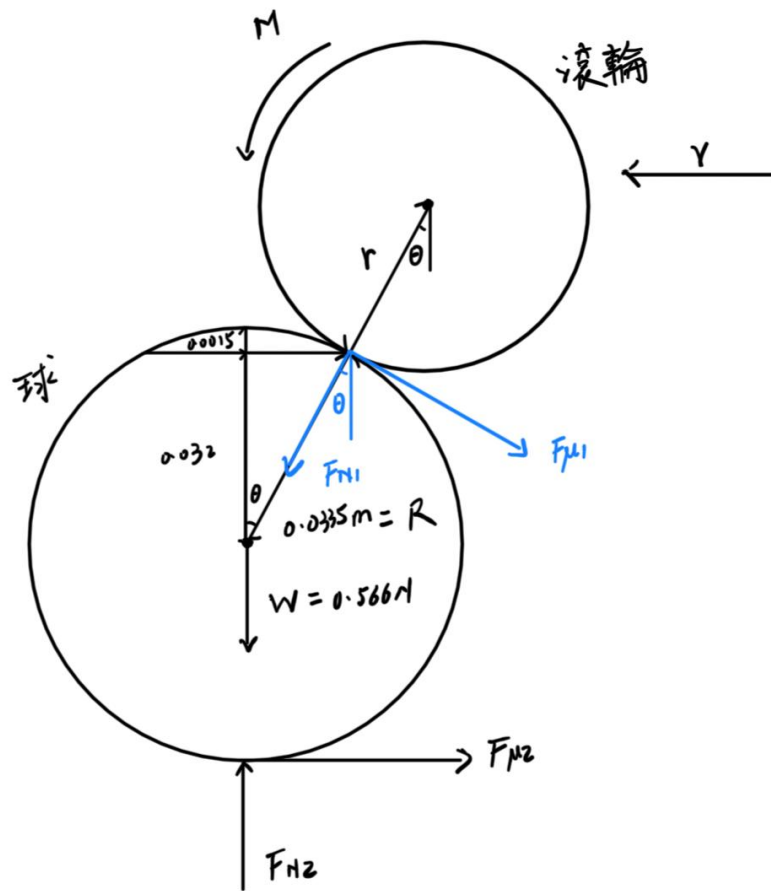
網球與地面摩擦係數: 0.046

滾輪直徑: 50mm

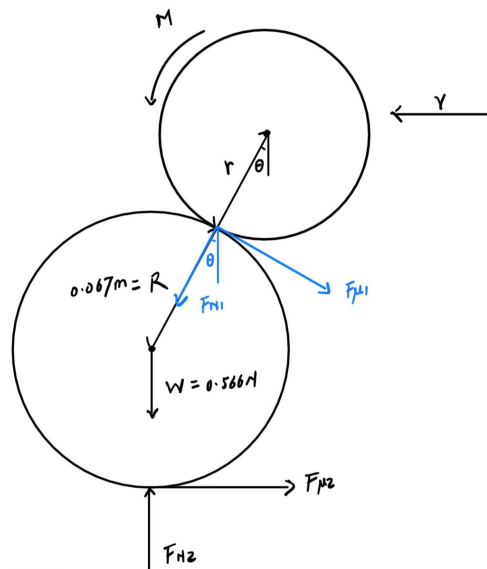
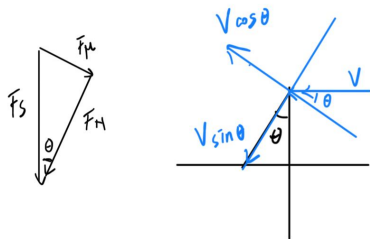
滾輪與網球間摩擦係數: 0.7

機器運行速度: 1.35m/s

滾輪軸心與地面距離: 86.7mm



上滾輪撿球機構計算



$$m = 0.0377 \text{ kg}, \quad k = 12550 \text{ N/m}, \quad \delta_2 = 3 \text{ mm}, \quad V = 1.35 \text{ m/s}$$

$$F_{N1} = F_d + F_s \cos \theta = 46.719 \text{ N}$$

$$\frac{1}{2} m V^2 = \frac{1}{2} k \delta_1^2 \Rightarrow m V^2 = k \delta_1^2 \Rightarrow \delta_1 = V \sqrt{\frac{m}{k}}$$

$$F_d = k \times \delta_1 = k \times V \sqrt{\frac{m}{k}} = V \sin \theta \sqrt{m k} = 10.749 \text{ N}$$

$$F_s = k \times \delta_2 = 37.65 \text{ N}$$

$$\cos^{-1} \left(\frac{32}{37.65} \right) = 17.21^\circ = \theta$$

$$F_{N2} = W + F_{N1} \cos \theta = 45.187 \text{ N}$$

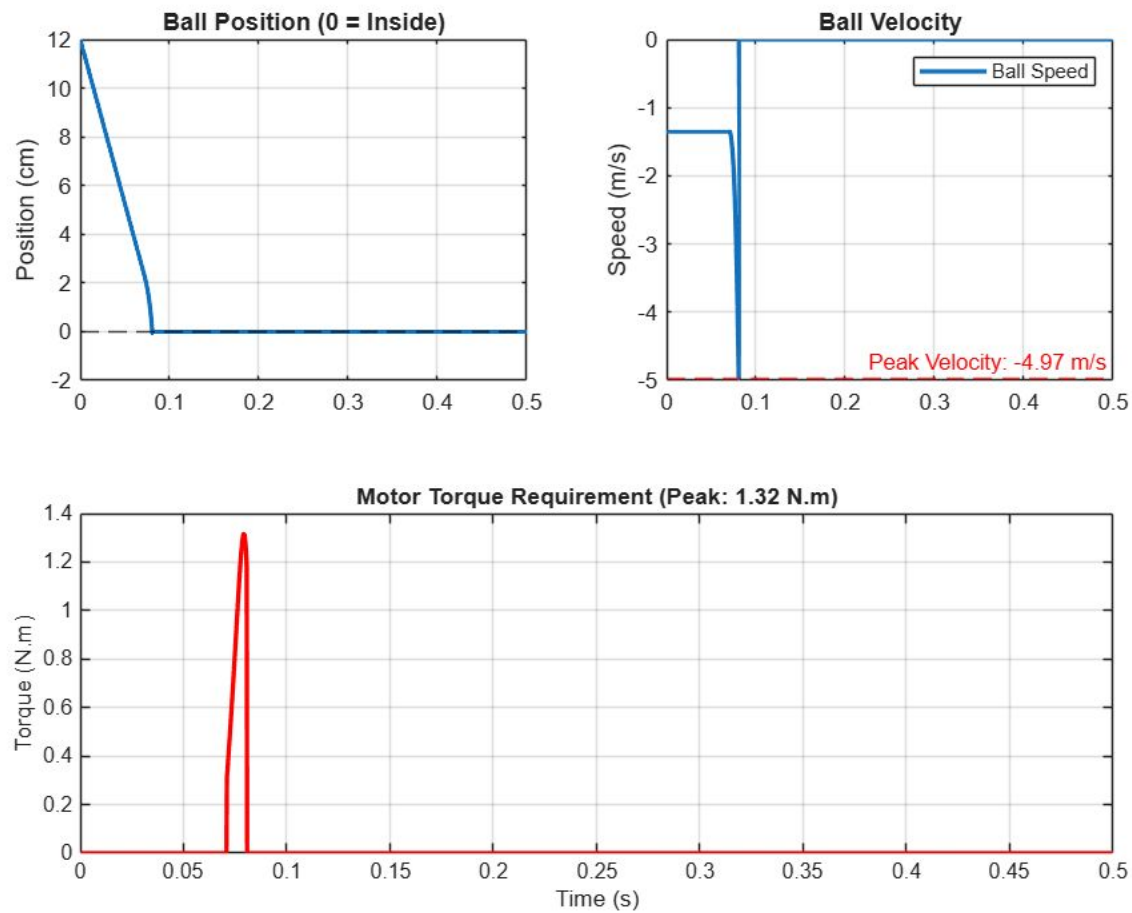
$$\begin{aligned} \rightarrow F_x: (F_{H1} + F_s \sin \theta) \cos \theta + F_{H2} &\geq F_{N1} \sin \theta \Rightarrow 47.955 > 13.82 \\ \mu_1 &= 0.7, \quad \mu_2 = 0.046 \end{aligned}$$

$$F_{H1} = F_{N1} \times \mu_1 = 0.7 F_{N1} = 32.7 \text{ N}$$

$$F_{H2} = F_{N2} \times \mu_2 = 0.046 F_{N2} = 2.079 \text{ N}$$

$$M = (F_{H1} + F_s \sin \theta) \cos \theta \times r = 1.047 \text{ N} \cdot \text{m}$$

$$V \cos \theta = 1.29 \text{ m/s} \Rightarrow \frac{60 \times 1.29}{\pi \times 0.05} = 493 \text{ rpm}$$



側向滾輪計算

標準網球直徑: 67mm

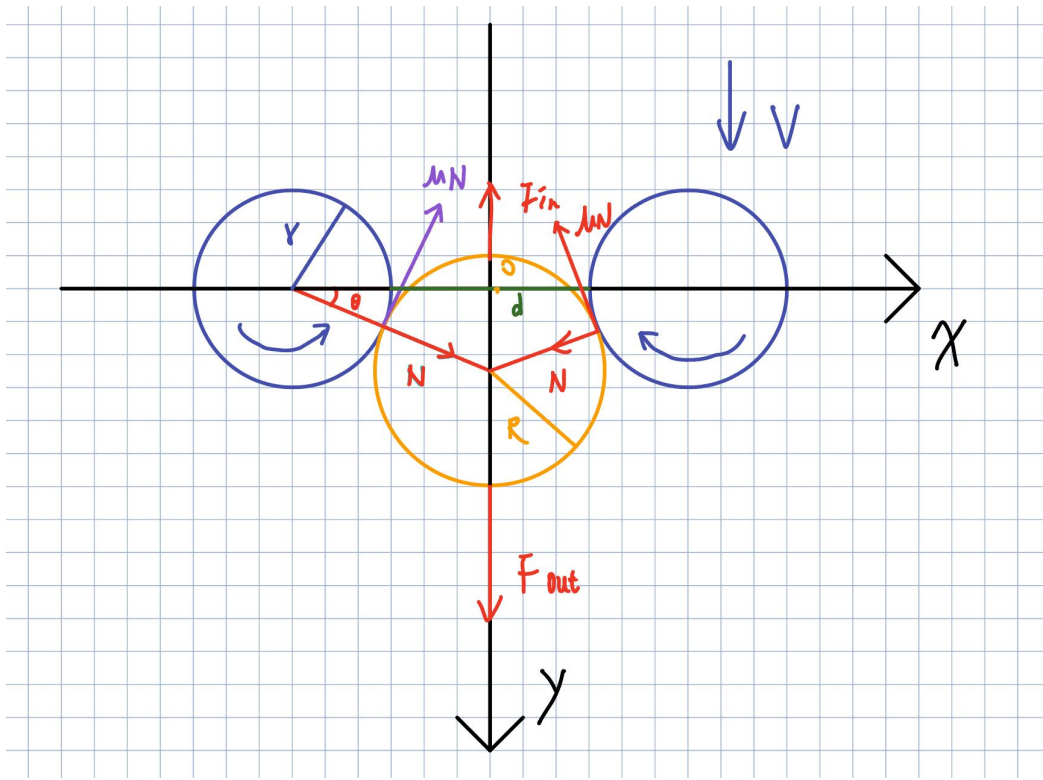
滾輪直徑: 50mm

滾輪間距: 56mm

標準網球重量: 57.7g

滾輪與網球間摩擦係數: 0.7

網球與地面摩擦係數: 0.046



$$F_{in} = \mu N \cos \theta$$

$$F_{out} = N \sin \theta$$

$$\text{球不彈出} \rightarrow F_{in} \geq F_{out}$$

$$\Rightarrow \mu N \cos \theta \geq N \sin \theta$$

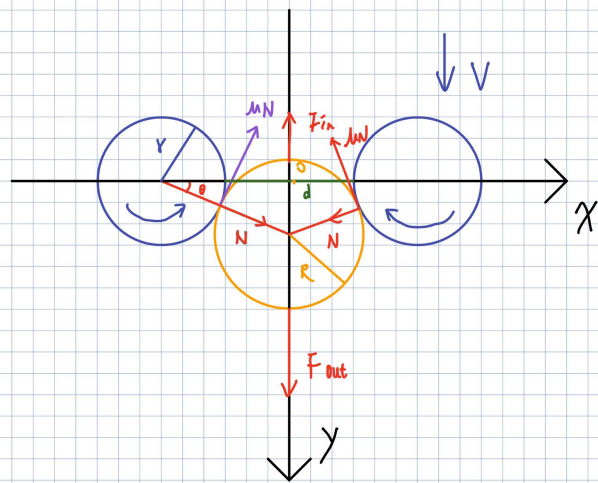
$$\Rightarrow \mu \geq \tan \theta$$

$$\mu = 0.7 \quad \tan \theta = \frac{\sqrt{(r+R)^2 - (r+\frac{d}{2})^2}}{r+\frac{d}{2}} = 0.44 \quad (\text{不會彈出})$$

r : 滾輪半徑

R : 網球半徑

d : 滾輪間距



$$W = \frac{d}{2} + r$$

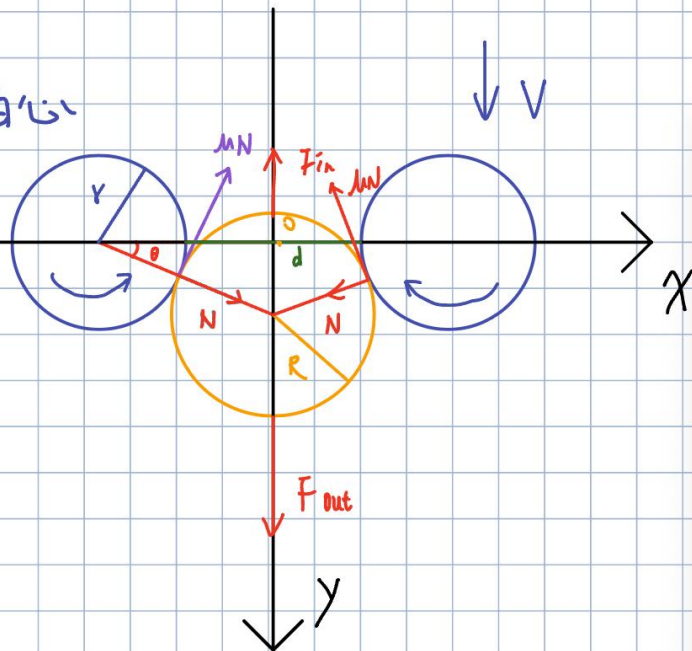
零坐標點到滾輪軸心

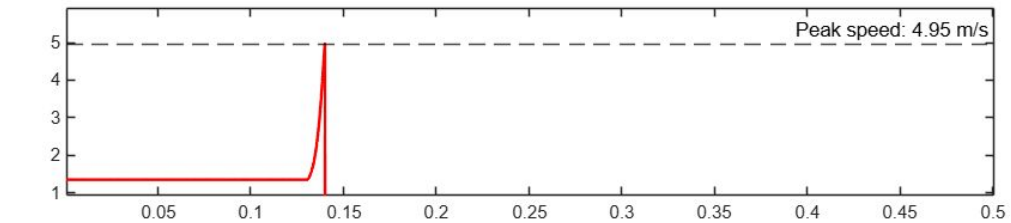
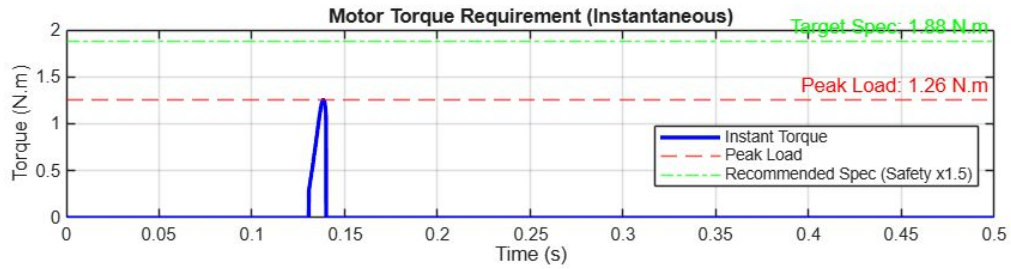
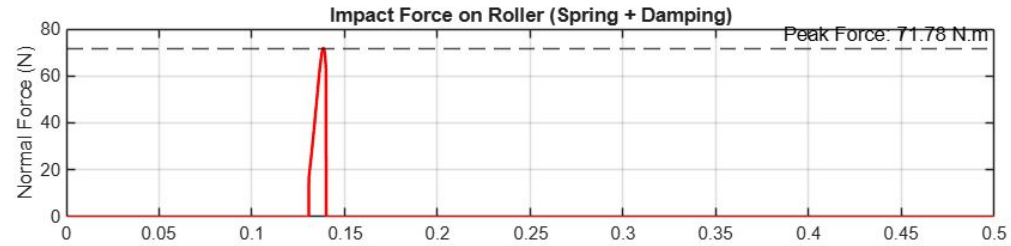
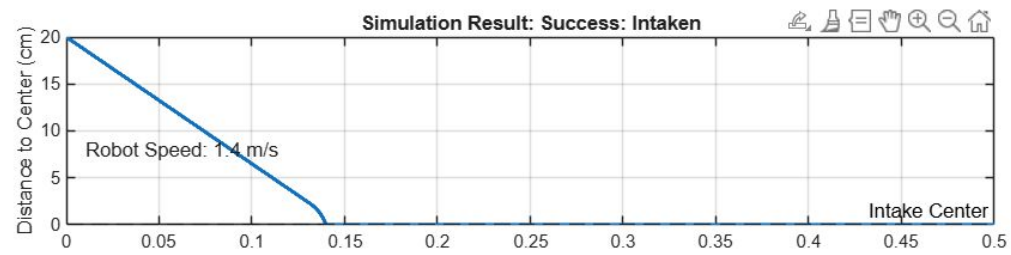
$$L = \sqrt{W^2 + y^2}$$

球心到滾輪軸心

$$\delta = r + R - L$$

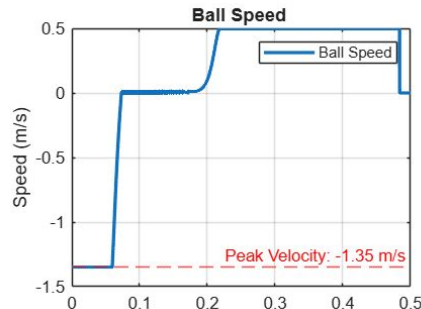
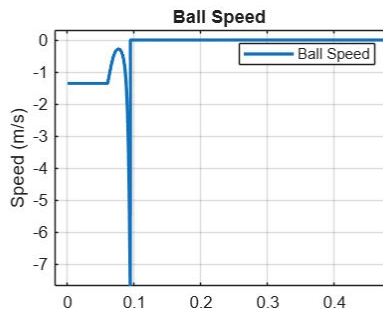
$$N = k\delta$$



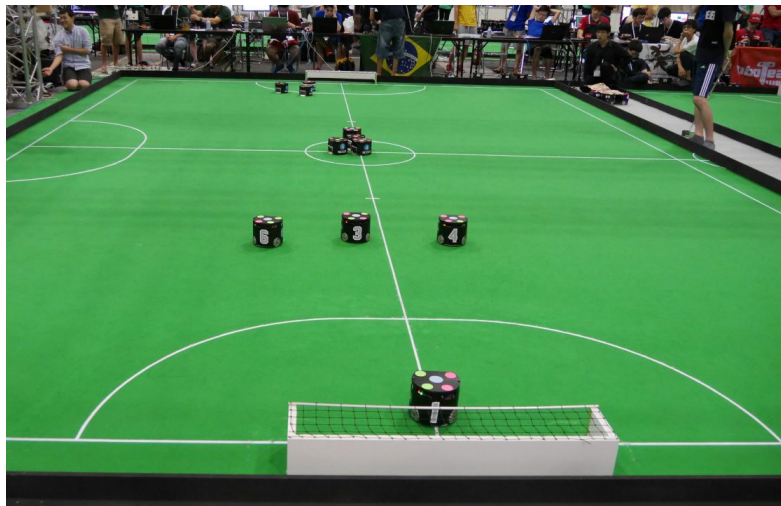


選擇側向滾輪的理由

- 受力分散於兩側，機構較不容易損壞
- 上側滾輪會受地面摩擦力影響，側向則不會
- 上側滾輪容易因地形起伏受影響，如果滾輪太低，球會直接彈飛(如果用可動的懸臂會導致進球速度不太一樣)
- 機構設計較簡單



視覺系統架構選擇: Global vs. On-board Vision



	Global Vision	On-board Vision
座標系統	絕對座標	相對座標
場地需求	需先架設	無
主要挑戰	架設困難、遮擋	運算負載高 (SLAM)

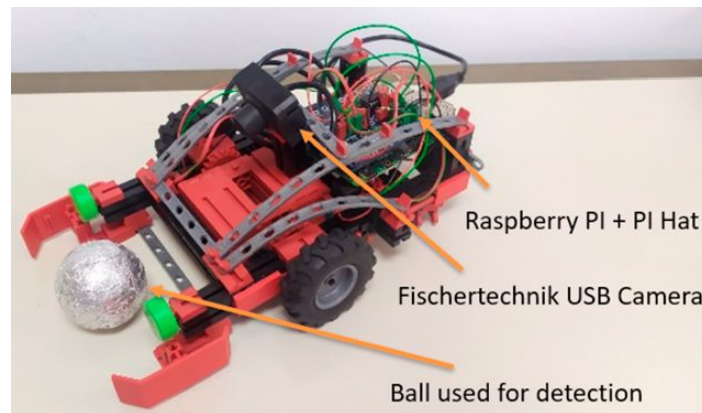
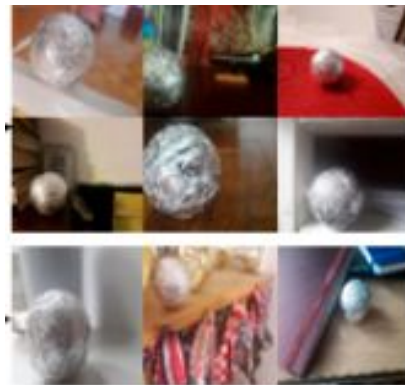
文獻背景

標題: Ball Detection Using Deep Learning
Implemented on an Educational Robot Based on
Raspberry Pi

出處: Sensors (2023), Keča et al.

實驗設計: 使用鋁箔紙製作銀色球, 拍攝1200張照片
採用70%(訓練) - 15%(驗證) - 15%(測試), 手動
模糊180張照片作為增強測試

在Raspberry Pi 比較 Hough Transform (霍夫圓形
變換)、U-NET (語義分割)、Faster R-CNN (物件偵
測)



影像辨識評估指標定義

綜合準確度 DICE

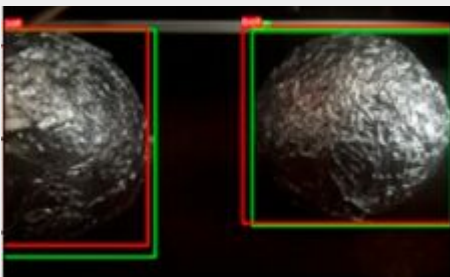
平衡誤判與漏抓

確保機器人不抓雜物
& 不漏看球

定位精準度 (IoU)

框得準確

決定撿球機構成功率



推論時間 (Inference Time)

處理一張照片時間

Hough Transform 參數僵化

手動參數依賴：需人工設定dp、(p1, p2)、minR, maxR 與最小距離 (minDist)

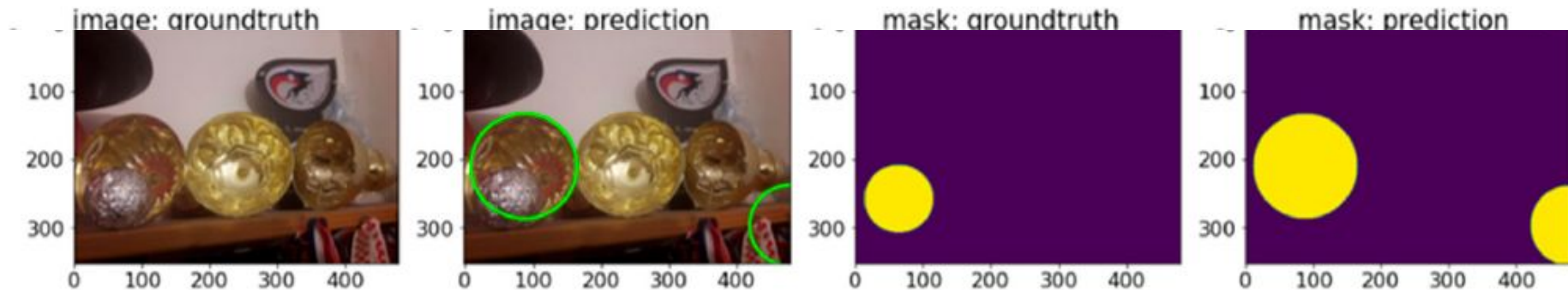
準確度崩盤：實驗顯示，無論如何微調，DICE 分數最高僅 0.16，遠低於深度學習的 0.9+

Hough	dp	p1, p2	minR, maxR	minDist	DICE (TEST)	IoU (TEST)	DICE (TEST AUG)	IoU (TEST AUG)
Hough 40-110-150	1.00	40, 110	5, 150	150	0.1193	0.0634	0.1642	0.0895
Hough 40-90-50	1.00	40, 90	5, 150	50	0.1538	0.0833	0.1316	0.0704
Hough 40-90-150	1.00	40, 90	5, 150	150	0.147	0.0793	0.1392	0.0748
Hough 50-40-50	1.2	50, 40	25,300	50	0.134	0.0718	0.1462	0.0788

Hough Transform 環境失效與動態模糊

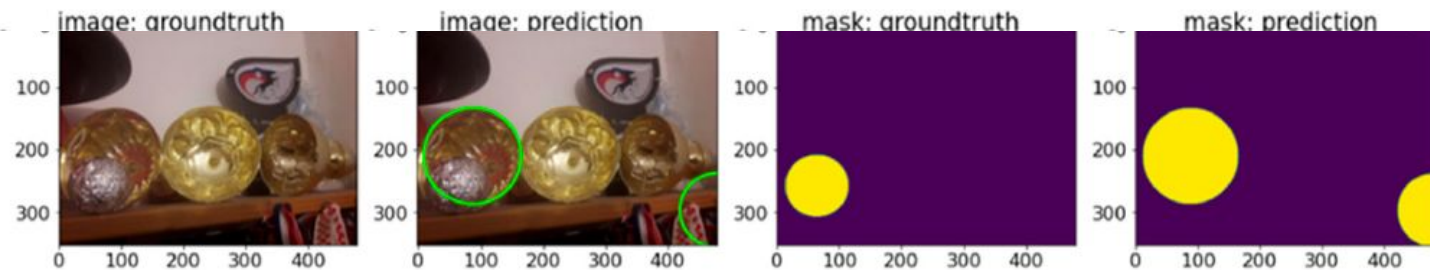
光影干擾 (Illumination): 文獻 Figure 10 證實, 單純依賴灰階 (Grayscale) 與邊緣特徵, 在陰影下極易失效 (False Negative), 或將圓形背景雜訊誤判為球 (False Positive)

形狀變異 (Deformation): 由於撿球機移動中拍攝網球可能為橢圓形或長條形狀

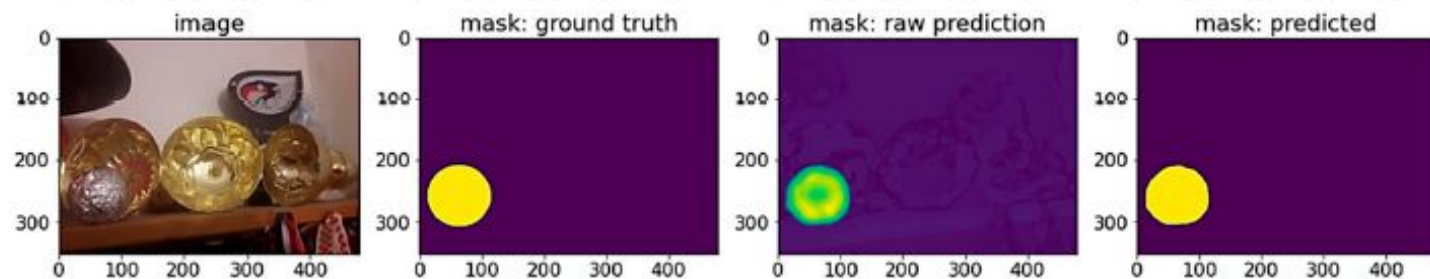


視覺辨識實際案例比較

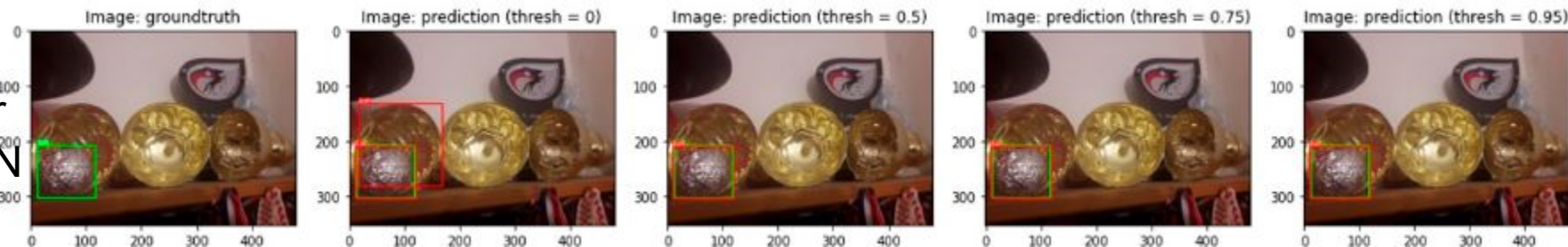
Hough



U-NET



Faster
R-CNN



U-NET, Faster R-CNN 實驗數據

Encoder	DICE (TEST)	IoU (TEST)	DICE (TEST AUG)	IoU (TEST AUG)
EFFICIENTNET-B0	0.9659	0.9341	0.828	0.7065
EFFICIENTNET-B1	0.967	0.9361	0.8583	0.7518
EFFICIENTNET-B2	0.9637	0.9299	0.8435	0.7294
EFFICIENTNET-B3	0.9658	0.9338	0.8374	0.7203
EFFICIENTNET-B4	0.9637	0.93	0.8041	0.6724

上: U-NET; 下: Faster R-CNN

Dice和IoU的實驗數據明顯比Hough Transform的結果好

CNN Backbone Model	DICE (TEST)	IoU (TEST)	DICE (TEST AUG)	IoU (TEST AUG)
RESNET50	0.9502	0.905	0.7422	0.59
MOBILENET_V3_LARGE	0.9621	0.9269	0.7683	0.6238
MOBILENET_V3_LARGE_320	0.9585	0.9203	0.8041	0.6724

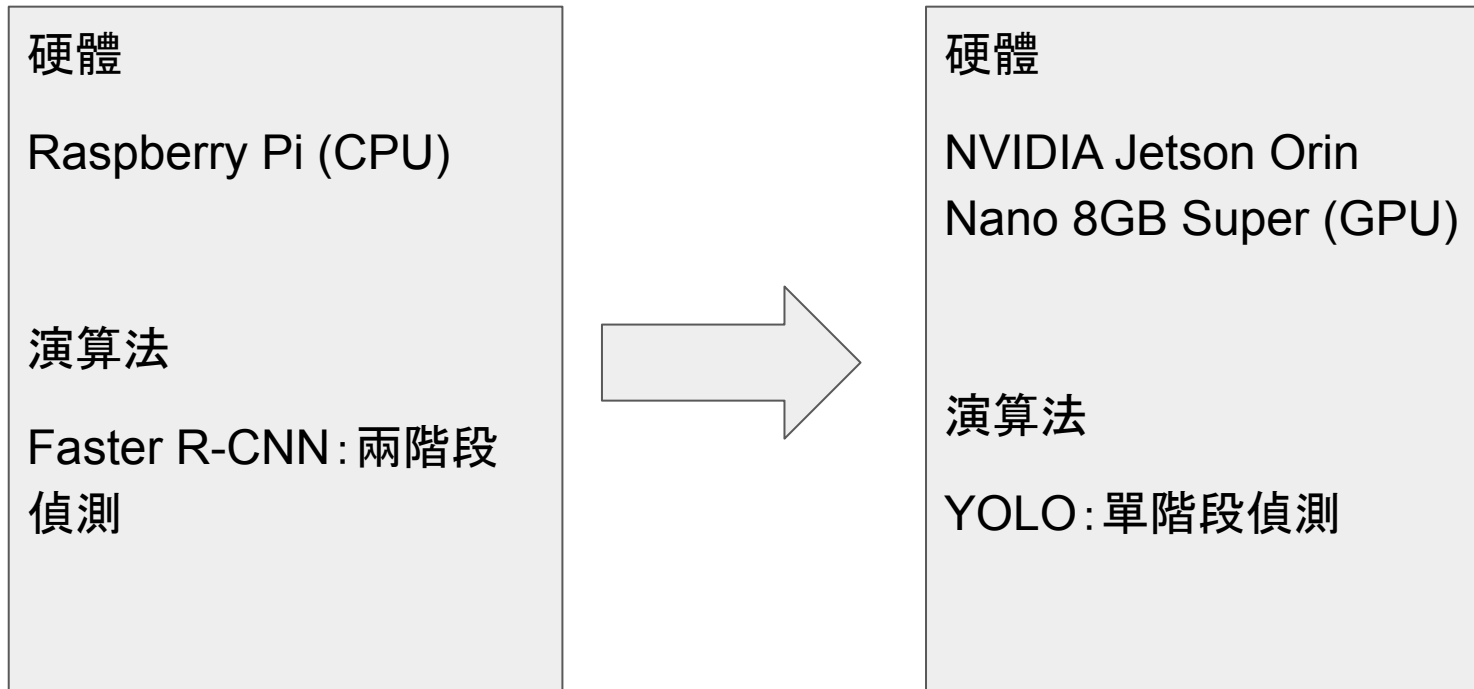
影像辨識運算時間

樹莓派算力瓶頸

網球機器人速度快，我們至少需要 15-20 FPS，純 CPU 算力缺口巨大。

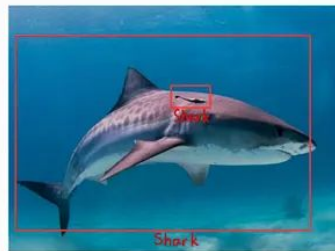
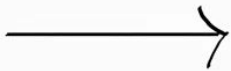
演算法	模型架構	Time / Image (秒/張)	FPS (幀數)	狀態
Faster R-CNN	ResNet50	69.79 s	0.01	不可用
U-NET	EfficientNet-B0	8.23 s	0.12	嚴重延遲
Object Det.	MobileNet_V3	2.01 s	0.5	過慢
Hough	Traditional	0.24 s	4.1	快但不準

解決方案提案

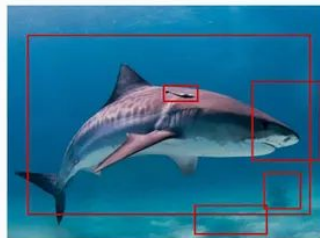


為什麼選擇 YOLO?

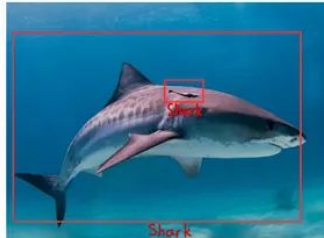
One-Stage detector



Two-Stage detector



region proposals



YOLO:

One-stage

即時性強(Real-time)

專為動態物件設計，
適合撿球機器人。

為什麼選擇 Jetson Orin Nano 8GB Super？

比較項目	Raspberry Pi 4 (文獻對照組)	Jetson Orin Nano 8GB Super (提案選擇)	勝出原因 (Why?)
AI 算力 (TOPS)	< 1 TOPS (純 CPU)	67 TOPS (GPU)	70倍效能差異
核心架構	ARM CPU (通用運算)	NVIDIA Ampere GPU (AI 專用)	支援 CUDA 與 TensorRT 硬體加速
多工負載能力	單工極限：跑 YOLO 滿載	多工餘裕：YOLO 佔部分算力，保留資源給 SLAM。	這是關鍵！ 機器人還要導航。
記憶體頻寬	LPDDR4 (較慢)	8GB LPDDR5 (102 GB/s)	處理高畫質影像與大型地圖必備
推論速度 (FPS)	0.5 FPS (MobileNet)	預估 > 30 FPS (YOLOv8)	達到即時追蹤網球的門檻

飆機器人官網

攝影鏡頭套件參考



Jetson Orin Nano Super 8GB 開發套件

SUB/official version optional

電源供應器 天線

256GB SSD (內含系統映像檔) 已安裝無線網卡 IMX219 相機模組 (77° 視角) 應克力外殼 (可安裝天線 附相機支架)

攝影鏡頭套件

Jetson Orin Nano **Super** (8GB s)

Super 67TOPS



Jetson Orin Nano Super 16GB 開發套件

SUB/official version optional

電源供應器 天線

256GB SSD (內含系統映像檔) 已安裝無線網卡 IMX219 相機模組 (77° 視角) 應克力外殼 (可安裝天線 附相機支架)

攝影鏡頭套件

Jetson Orin NX **Super** (16GB s)

Super 157TOPS

未來計畫

1. 用數學方式決定相機規格、架設高度、角度
2. 路徑規畫相對不相關未來再做選擇
3. SLAM 建圖功能

Thanks for watching